## CKA Exam Domains


- Domain I: Troubleshooting (30%) - The High-Stakes Domain
- Domain II: Cluster Architecture, Installation & Configuration (25%) - The Modern Tooling Domain
- Domain III: Services & Networking (20%) - The Connectivity Domain
- Domain IV: Workloads & Scheduling (15%) - The Resource Management Domain
- Domain V: Storage (10%) - The Persistence Domain


## Domain I: Troubleshooting (30%) - The High-Stakes Domain

As the most heavily weighted domain, Troubleshooting presents the most complex and challenging scenarios. Success in this area is paramount to passing the exam.

**Core Task 1:** Control Plane Failure Diagnosis. A recurring, high-difficulty scenario involves diagnosing and repairing a non-functional control plane. The typical setup presents a cluster where key components like kube-apiserver and kube-scheduler are failing, while etcd and the kubelet remain operational. The task requires a systematic approach to debugging. Candidates must inspect the static pod manifests located in /etc/kubernetes/manifests, validate the YAML configuration files for errors, and examine component logs. Since these components run as static pods, standard kubectl logs commands will not work; proficiency with node-level tools like journalctl or container-runtime-specific commands (e.g., crictl logs) is essential.

**Core Task 2:** Worker Node & CNI Misconfiguration. This pattern tests the candidate's understanding of the networking stack at the node level. A common task involves installing a Container Network Interface (CNI) plugin, such as Calico or Flannel, from a provided manifest file. After installation, pods remain in a Pending or ContainerCreating state. The root cause is often a mismatch between the Pod CIDR range configured for the cluster via kubeadm and the network range defined in the CNI's configuration, which is typically stored in a ConfigMap or a Custom Resource. The candidate must identify this discrepancy and patch the CNI's configuration to align with the cluster's Pod CIDR, thereby restoring pod-to-pod connectivity. This directly assesses the "Troubleshoot clusters and nodes" competency outlined in the official curriculum.

**Core Task 3:** Application & Networking Debugging. These tasks present a functioning cluster but a failing application, often due to connectivity issues. Scenarios may involve debugging why a service is unreachable, which requires checking that the service's label selector correctly matches the pods' labels, verifying that the Endpoints object has been populated with pod IPs, and ensuring that no

Network Policies are inadvertently blocking traffic. Another common variant involves troubleshooting DNS resolution failures, which points towards investigating the health and configuration of CoreDNS in the kube-system namespace.

## Domain II: Cluster Architecture, Installation & Configuration (25%) - The Modern Tooling Domain

This domain has been significantly updated to include modern application management and cluster extension tools.

**Core Task 1:** Helm Operations. Proficiency with the Helm CLI is now mandatory. A representative task involves installing a software package (e.g., ArgoCD) using a Helm chart from a public repository. A frequent and subtle variation of this task requires installing a chart while preventing the installation of its associated CRDs, under the premise that the CRDs already exist in the cluster from a previous installation. This requires knowledge of specific Helm flags like --skip-crds or chart-specific values passed via --set crds.install=false. Another reported task involves using helm template to render a chart's manifests into a local file for inspection or modification before applying them to the cluster with kubectl apply -f.

**Core Task 2:** CRD Management. This tests the ability to interact with and understand cluster API extensions. A typical two-part task involves first listing all CRDs in the cluster that match a certain keyword (e.g., cert-manager) and redirecting the output to a file. The second part requires the candidate to use the kubectl explain command to introspect the schema of one of these CRDs. For example, the task might be to find and document the purpose of a specific field, such as spec.secretName, within a Certificate CRD. This is a direct, practical test of the "Understand CRDs" competency.

**Core Task 3:** RBAC Configuration. A classic CKA task, Role-Based Access Control configuration remains a core competency. Exam questions typically present a scenario requiring the creation of a Role (for namespace-scoped permissions) or a ClusterRole (for cluster-wide permissions). The candidate must then create a RoleBinding or ClusterRoleBinding to grant a specific user or ServiceAccount the defined permissions. The permissions are usually very specific, such as granting the ability to only get, list, and watch pods within the frontend namespace.

## Domain III: Services & Networking (20%) - The Connectivity Domain

Networking questions on the CKA exam are known for their complexity and nuance, requiring careful reading and precise execution.

**Core Task 1:** Ingress to Gateway API Migration. This is a flagship task of the 2025 curriculum, reflecting the industry's shift towards the more expressive and role-oriented Gateway API. A common scenario involves an existing application that is exposed to external traffic via a traditional Ingress resource. The candidate's task is to create a new Gateway resource and an HTTPRoute resource to replicate the exact functionality of the old Ingress, including host-based routing and TLS termination using an existing secret. After verifying the new configuration works, the final step is to delete the legacy Ingress resource. This task directly assesses the "Use the Gateway API to manage Ingress traffic" competency.

**Core Task 2:** Advanced Network Policy Implementation. Network Policy questions have evolved beyond simple ingress denial. A frequent format presents a complex traffic flow requirement and provides several pre-written YAML manifests for different Network Policies. The candidate must analyze the requirement (e.g., "Allow ingress traffic to pods with label app=backend only from pods in the frontend namespace on TCP port 8080") and select and apply the one manifest that correctly implements this rule. Success requires a deep understanding of how podSelector, namespaceSelector, and ipBlock work together in both ingress and egress rules.

**Core Task 3:** Service Exposure. While fundamental, this task appears frequently and tests for precision under pressure. The objective is typically to expose an existing Deployment using a NodePort service. The question will specify the exact port on the node (nodePort), the port on the service, and the container's target port that traffic should be forwarded to.

## Domain IV: Workloads & Scheduling (15%) - The Resource Management Domain

This domain focuses on the lifecycle of applications running on the cluster and the mechanisms for controlling their placement and resource consumption.

**Core Task 1:** Node Resource Allocation. This is a highly practical task that simulates a real-world resource contention problem. The scenario involves a Deployment with multiple replicas where some pods are stuck in the Pending state due to insufficient CPU or memory on the worker nodes. The candidate must first use kubectl describe node to inspect the node's Allocatable resources. Then, they must perform a calculation to determine how to distribute the available resources evenly among the required number of pods, ensuring to leave a small buffer for system daemons. Finally, they must

edit the Deployment manifest to set appropriate CPU and memory requests and limits that allow all replicas to be scheduled successfully.

**Core Task 2:** Horizontal Pod Autoscaler (HPA) Configuration. This is a common task testing the "Configure workload autoscaling" competency. The candidate is asked to create a HorizontalPodAutoscaler for an existing Deployment. The requirements are specific: scale based on CPU utilization, with a defined minimum and maximum number of replicas (e.g., min 2, max 5), and a target average CPU utilization percentage across all pods (e.g., 70%).

**Core Task 3:** Default StorageClass Configuration. This is a subtle administrative task that can easily trip up candidates unfamiliar with annotations. The scenario presents a cluster with multiple StorageClass objects, none of which is designated as the default. The task is to make one of them the default for all future dynamic provisioning requests that do not specify a storageClassName. This cannot be achieved by simply editing the object's spec. The correct method is to use kubectl patch to set the annotation storageclass.kubernetes.io/is-default-class to "true" on the desired StorageClass. It is also best practice to ensure this annotation is set to "false" or is absent on all other StorageClasses.

## Domain V: Storage (10%) - The Persistence Domain

While having the lowest weight, storage questions test specific and sometimes non-obvious administrative knowledge.

**Core Task 1**: Binding to an Existing Persistent Volume (PV). This task moves beyond simple dynamic provisioning and tests a deeper understanding of the PV/PVC binding lifecycle. The environment contains a pre-created PV with specific characteristics (e.g., a capacity of 1Gi, an access mode of ReadWriteOnce, and a particular storageClassName). The candidate must create a new PersistentVolumeClaim (PVC) whose specification (access mode, storage request, and storage class) precisely matches the existing PV, which will cause the control plane to bind them together. The final step is to create a pod that mounts this PVC.

**Core Task 2**: Default StorageClass Configuration. This is a subtle administrative task that can easily trip up candidates unfamiliar with annotations. The scenario presents a cluster with multiple StorageClass objects, none of which is designated as the default. The task is to make one of them the default for all future dynamic provisioning requests that do not specify a storageClassName. This cannot be achieved by simply editing the object's spec. The correct method is to use kubectl patch to set the annotation storageclass.kubernetes.io/is-default-class to "true" on the desired StorageClass. It is also best practice to ensure this annotation is set to "false" or is absent on all other StorageClasses.

# The CKA 2025 Practice Question Bank.

This question bank is designed to reflect the "brownfield," scenario-based nature of the 2025 CKA exam. Each question requires analysis, modification, or troubleshooting of a pre-existing environment.

## Domains with 10 questions each


- Domain: Troubleshooting (10 Questions)
- Domain: Cluster Architecture, Installation & Configuration (10 Questions)
- Domain: Services & Networking (10 Questions)
- Domain: Workloads & Scheduling (10 Questions)
- Domain: Storage (10 Questions)


## Domain: Troubleshooting (10 Questions)

## Question 1: Control Plane Component Failure

- **Scenario:** The cluster's API server is unresponsive. `kubectl` commands are failing with a connection refused error. The `etcd` and `kubelet` services on the control plane node `cp-node` are confirmed to be running.
- **Initial State:** The static pod manifest for the kube-apiserver located at `/etc/kubernetes/manifests/kube-apiserver.yaml` on `cp-node` has been corrupted. A command argument is misspelled.
- **Task:** SSH into `cp-node`. Investigate the kube-apiserver static pod manifest and correct the configuration error. Ensure the API server starts successfully and the cluster becomes responsive.
- **Validation:** From the base node, run `kubectl get nodes`. The command should execute successfully and show the cluster nodes.

### Solution

This scenario simulates a broken control plane where the `kube-apiserver` is down due to a syntax error in its static pod manifest. Since the API server is the entry point for all `kubectl`

commands, standard CLI tools will not work. Accessing the node via SSH and fixing the manifest file directly is the required approach.

**Step-by-Step Solution:**

1. **SSH into the Control Plane Node:** Access the node where the API server is running (specified as `cp-node`).

   `bash ssh cp-node`

2. **Verify the Component Status (Optional but Recommended):** Since `kubectl` is down, use `crictl` or check the kubelet logs to confirm the API server is failing.

   ```bash

   ## Check running containers (apiserver will likely be missing or exiting)

   crictl ps | grep apiserver

   ## Check kubelet logs for manifest errors

   journalctl -u kubelet | tail -n 50 ```

3. **Locate and Edit the Manifest:** The static pod manifests are managed by the kubelet and located in `/etc/kubernetes/manifests`.

   ```bash cd /etc/kubernetes/manifests/ ls

   ## You should see kube-apiserver.yaml

   ```

4. **Fix the Configuration Error:** Open the file and look for the misspelled argument.

   `bash vi kube-apiserver.yaml`

   *Look for a line that looks like:* `- --commmand-argument` (e.g., misspelled) or an invalid flag. *Correct it to:* `- --command-argument` (correct spelling).
   ```

*Example:* finding a typo like `--etcd-servers` written as `--etcd-serveer`.

5. **Save and Exit:** Save the file (`:wq` in vi). The kubelet automatically watches this directory and will detect the change. It will kill the old (failing) pod and start a new one with the corrected configuration.

6. **Verify Recovery:** Wait a few moments for the API server to initialize.

   ```bash

   ## Check if the container is running now

   crictl ps | grep apiserver ```

7. **Exit the Node:**

   `bash exit`

**Validation:**

Back on the base node, try to interact with the cluster.

```
kubectl get nodes
```

*Output should list the nodes with Status 'Ready'.*

---

## Question 2: CNI Pod CIDR Mismatch

- **Scenario:** A new worker node, `worker-3`, has been added to the cluster, but pods scheduled on it cannot communicate with pods on other nodes. Pods on `worker-3` are stuck in the `ContainerCreating` state.
- **Initial State:** The cluster is using the Calico CNI. The main Calico configuration ConfigMap, named `calico-config` in the `kube-system` namespace, defines a Pod IP range that does not include the Pod CIDR assigned to `worker-3`.
- **Task:** Identify the Pod CIDR assigned to `worker-3` by inspecting the node object. Edit the `calico-config` ConfigMap in the `kube-system` namespace to correctly include the Pod CIDR range of all nodes.
```

- **Validation:** Create a test pod and ensure it gets scheduled on `worker-3` and enters the Running state. Exec into the pod and ping another pod on a different worker node.

### Solution

This scenario tests your understanding of Kubernetes networking, specifically the relationship between the Node's `PodCIDR` (assigned by the Kubernetes Controller Manager) and the overlay network configuration managed by the CNI plugin (Calico). If the CNI is not aware of the CIDR range intended for a node, it cannot properly assign IPs or route traffic, leading to connectivity issues or pods stuck in `ContainerCreating`.

**Step-by-Step Solution:**

1. **Identify the Node's Assigned Pod CIDR:** Check the `spec.podCIDR` field of the problematic node (`worker-3`).

   ```bash kubectl get node worker-3 -o jsonpath='{.spec.podCIDR}'

   ## Example Output checking: 192.168.3.0/24

   ---

   ```

2. **Inspect the CNI Configuration:** Check the `calico-config` ConfigMap in the `kube-system` namespace to see the currently configured IP pool.

   `bash kubectl get cm calico-config -n kube-system -o yaml`

   *Look for a field defining the network range, such a `typha_service_name` or a JSON blob in the `data` section describing the IP Pool (e.g., `cni_network_config`). Assume the configured range is too narrow (e.g., `192.168.0.0/23`) and misses the new node's CIDR.*

3. **Update the ConfigMap:** Edit the ConfigMap to expand the range to cover the new node's CIDR (e.g., change `192.168.0.0/23` to `192.168.0.0/16` or explicitly add the missing range).

   `bash kubectl edit cm calico-config -n kube-system`

   *Locate the CIDR definition within the data section and update it. Save and exit (`:wq`).*

4. **Restart CNI Pods:** For the changes to take effect, restart the Calico node pods (DaemonSet).

   `bash kubectl rollout restart daemonset calico-node -n kube-system`

**Validation:**

1. **Create a Test Pod on the Node:** Use a `nodeSelector` or `nodeName` to force a pod onto `worker-3`.

   `bash kubectl run test-pod --image=busybox --restart=Never --overrides='{"spec": {"nodeName": "worker-3"}}' -- sleep 3600`

2. **Verify Pod Running Status:**

   ```bash kubectl get pod test-pod

   ## Status should eventually change to Running

   ---

   ```

3. **Test Connectivity:** Ping a pod on a different node.

   ```bash

   ## Get IP of a pod on another node (e.g., coredns)

   ---

   kubectl get pods -n kube-system -o wide

   ## Ping from test-pod

   ---

   kubectl exec -it test-pod -- ping -c 2 ```

---

## Question 3: CoreDNS Configuration Error

- **Scenario:** Pods across the cluster are unable to resolve external domain names (e.g., `google.com`), although internal service name resolution is working.
- **Initial State:** The `coredns` ConfigMap in the `kube-system` namespace has been modified, and the `forward` plugin, which points to upstream DNS servers, has been accidentally removed.
   ```

- **Task:** Edit the `coredns` ConfigMap. Re-add the `forward` plugin to the Corefile configuration, pointing to a public DNS server like `8.8.8.8`. After saving the changes, restart the CoreDNS pods to apply the new configuration.
- **Validation:** Exec into a busybox pod and run `nslookup google.com`. The command should succeed.

## Solution

This scenario simulates a common DNS issue where the cluster can resolve internal services (like `kubernetes.default`) but fails to resolve external domains (like `google.com`). This is controlled by the `forward` plugin in the CoreDNS configuration (Corefile). If this plugin is missing or misconfigured, CoreDNS doesn't know where to send queries for domains it doesn't authorize.

**Step-by-Step Solution:**

1. **Verify the Problem (Optional):** Check the current CoreDNS configuration to confirm the missing `forward` plugin.

   `bash kubectl get cm coredns -n kube-system -o yaml`

   *You should see the `Corefile` data block. Notice that the `forward . ...` line is missing.*

2. **Edit the ConfigMap:** You need to add the `forward` plugin back into the Corefile.

   `bash kubectl edit cm coredns -n kube-system`

   *Locate the `Corefile` section. Add the `forward` line inside the main block (usually the one responsible for `.`). It should look something like this:*

   `text .:53 { errors health { lameduck 5s } ready kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure fallthrough in-addr.arpa ip6.arpa ttl 30 } # Add this line: forward . 8.8.8.8 # OR use the node's resolver: # forward . /etc/resolv.conf cache 30 loop reload loadbalance }`

   *Save and exit (`:wq`).*

3. **Restart CoreDNS Pods:** ConfigMap changes are not picked up immediately by running pods (unless they have a reloader, but explicit restart is faster and safer for the exam).

   `bash kubectl rollout restart deployment coredns -n kube-system`

**Validation:**

1. **Run a Test Pod:** Launch a temporary pod with DNS tools (busybox or dnsutils).

   `bash kubectl run test-dns --image=busybox:1.28 --restart=Never --rm -it --nslookup google.com`

   *(Note: busybox 1.28 is often recommended because newer versions sometimes have issues with nslookup).*

   *Output should show the address of google.com.*

---

## Question 4: Ingress Controller Pods Not Ready

- **Scenario:** The NGINX Ingress Controller pods in the `ingress-nginx` namespace are in a `CrashLoopBackOff` state. Ingress resources are not functioning.
- **Initial State:** The `ingress-nginx-controller` Deployment has a misconfigured liveness probe with an incorrect port, causing the kubelet to repeatedly kill and restart the pods.
- **Task:** Inspect the `ingress-nginx-controller` Deployment in the `ingress-nginx` namespace. Identify the incorrect port in the liveness probe configuration and correct it to match the health check port exposed by the controller (port `10254`).
- **Validation:** Run `kubectl get pods -n ingress-nginx`. The controller pods should enter the Running state and remain stable.

### Solution

This scenario tests your ability to troubleshoot `CrashLoopBackOff` errors caused by misconfigured health checks. A `livenessProbe` determines if a container is running. If it fails, the kubelet kills the container and restarts it. If the probe is checking the wrong port, it will continually fail even if the application is healthy, creating a restart loop.

**Step-by-Step Solution:**

1. **Identify the Problematic Pods:** Check the pods in the specified namespace.

   `bash kubectl get pods -n ingress-nginx`

   *You should see pods with status `CrashLoopBackOff` or high restart counts.*

2. **Investigate the Cause:** Describe one of the failing pods to look at the Events section.

   `bash kubectl describe pod <pod-name> -n ingress-nginx`

*Look for "Liveness probe failed: Connection refused" or similar messages in the Events. Note the port being successfully or unsuccessfully accessed.*

3. **Edit the Deployment:** The pods are managed by a Deployment. You must edit the Deployment manifest.

   `bash kubectl edit deployment ingress-nginx-controller -n ingress-nginx`

4. **Correct the Liveness Probe:** Scroll down to the `livenessProbe` section of the container spec. *Change the port field from the incorrect value (e.g., `8080` or `80`) to the correct port `10254`.*

   `yaml livenessProbe: httpGet: path: /healthz port: 10254 # <-- Ensure this is 10254 scheme: HTTP initialDelaySeconds: 10 periodSeconds: 10`

   *Save and exit (`:wq`).*

5. **Verify Verification:** The Deployment will rollout a new ReplicaSet with the fixed configuration.

   `bash kubectl get pods -n ingress-nginx -w`

   *Watch as the old pods terminate and the new pods reach the `Running` state and stay there (Ready 1/1).*

---

## Question 5: Node NotReady due to Kubelet Issue

- **Scenario:** A worker node named `node-fail` is reporting a `NotReady` status. Pods are being evicted from this node.
- **Initial State:** The kubelet service on `node-fail` has stopped due to a configuration error in `/var/lib/kubelet/config.yaml` (e.g., an invalid `cgroupDriver` value).
- **Task:** SSH into `node-fail`. Check the status of the kubelet service using `systemctl status kubelet`. Examine the service logs using `journalctl -u kubelet` to identify the configuration error. Correct the error in `/var/lib/kubelet/config.yaml` and restart the kubelet service.
- **Validation:** From the control plane, run `kubectl get nodes`. The status of `node-fail` should return to `Ready`.

### Solution

This question tests your ability to troubleshoot node-level components, specifically the `kubelet`, which is the primary node agent. The `kubelet` is a systemd service, so you must use standard Linux system administration tools (`systemctl`, `journalctl`, `vi`) rather than `kubectl`.

**Step-by-Step Solution:**

1. **SSH into the Failing Node:** Access the node experiencing issues (specified as `node-fail` or similar).

   `bash ssh node-fail`

2. **Check Kubelet Status:** Verify if the service is running, failed, or stopped.

   `bash systemctl status kubelet`

   *You will likely see `Active: activating (auto-restart)` or `Active: failed`. The exit code might be non-zero.*

3. **Inspect Logs for Errors:** Check the logs to find the *reason* it is failing.

   `bash journalctl -u kubelet | tail -n 50`

   *Look for specific error messages. A common one in this scenario is related to the **cgroup driver**. For example: "failed to run Kubelet: misconfiguration: kubelet cgroup driver: "systemd" is different from docker cgroup driver: "cgroupfs"".*

   *Another possibility is a syntax error in the config file, which the logs will pinpoint.*

4. **Edit the Configuration:** The logs likely pointed to `/var/lib/kubelet/config.yaml`. Open it.

   `bash sudo vi /var/lib/kubelet/config.yaml`

   *Find the erroneous parameter. If the error was the cgroup driver mismatch, change `cgroupDriver: systemd` to `cgroupDriver: cgroupfs` (or vice versa, matching what the container runtime is using).*

5. **Restart the Kubelet:** After saving the configuration, you must restart the service for changes to take effect.

   `bash sudo systemctl daemon-reload # Optional, but good practice if unit file changed sudo systemctl restart kubelet`

6. **Verify Node Recovery (On Node):**

`bash systemctl status kubelet`

*Ensure it says `Active: active (running)`.*

*Exit the node:*

`bash exit`

7. **Verify Node Recovery (From Control Plane):** Back on the main terminal: `bash kubectl get nodes` *The node status should change from `NotReady` to `Ready`. (It may take up to a minute).*

---

## Question 6: Service Endpoint Failure

- **Scenario:** A service named `frontend-svc` in the `default` namespace exists, but it has no endpoints, even though there are running pods that should be part of the service.
- **Initial State:** The `frontend-svc` Service has a selector `app=frontend-app`, but the corresponding Deployment's pods have the label `app=frontend-web`.
- **Task:** Modify the selector of the `frontend-svc` Service to match the labels of the existing pods (`app=frontend-web`).
- **Validation:** Run `kubectl describe service frontend-svc`. The Endpoints field should now be populated with the IP addresses of the running pods.

### Solution

Services find the pods they route traffic to using **Label Selectors**. If the Service's `spec.selector` does not exactly match the key-value pairs in the Pod's `metadata.labels`, the `Endpoints` list will be empty, and no traffic will reach the application.

**Step-by-Step Solution:**

1. **Verify the Issue:** Check the service and note the empty Endpoints.

   `bash kubectl describe service frontend-svc`

   *Look for the `Endpoints:` row. It will likely say `<none>`. Also note the `Selector:` value (e.g., `app=frontend-app`).*

2. **Check Pod Labels:** Find the actual labels on the application pods.

`bash kubectl get pods --show-labels`

*Locate the frontend pods and see their labels (e.g., `app=frontend-web`).*

3. **Edit the Service:** Update the service to match the pods.

   `bash kubectl edit service frontend-svc`

4. **Update the Selector:** Find the `selector` section and change the value. Change `app: frontend-app` to `app: frontend-web`.

   `yaml spec: selector: app: frontend-web # Updated to match pods ports: ...`

   *Save and exit (`:wq`).*

5. **Validation:** Check the endpoints again. `bash kubectl describe service frontend-svc` *The `Endpoints` field should now show IP addresses (e.g., `10.244.1.5:80`).*

---

## Question 7: Persistent Volume Claim Stuck in Pending

- **Scenario:** A developer has created a PersistentVolumeClaim named `db-pvc` that is stuck in the `Pending` state and will not bind.
- **Initial State:** The `db-pvc` requests 5Gi of storage with the `standard` StorageClass. However, the `standard` StorageClass provisions volumes that use the `ReadWriteMany` access mode, while the PVC is requesting `ReadWriteOnce`.
- **Task:** There are no PVs that can satisfy the claim. Edit the `db-pvc` PersistentVolumeClaim and change its requested `accessModes` from `ReadWriteOnce` to `ReadWriteMany` to match what the StorageClass provides.
- **Validation:** Run `kubectl get pvc db-pvc`. The status should change from `Pending` to `Bound`.

### Solution

For a PersistentVolumeClaim (PVC) to bind to a PersistentVolume (PV), the request must be satisfied by the volume's capabilities. This includes capacity, storage class, and **Access Modes**. If a PVC requests `ReadWriteOnce` but the available PVs (or the dynamic provisioner) only support `ReadWriteMany`, binding will fail.

**Step-by-Step Solution:**

1. **Investigate the PVC:** Check why the PVC is pending.

   `bash kubectl describe pvc db-pvc`

   *Check the Events section. You might see messages like "waiting for a volume to be created, either by external provisioner..." or "no persistent volumes available for this claim and no storage class is set".*

2. **Edit the PVC:** You need to change the `accessModes`. **Important:** You typically cannot update `accessModes` of an existing PVC if it has already been bound or if the field is immutable (depending on the K8s version and state). However, for a *Pending* PVC that hasn't found a match, you often can, or you may need to delete and recreate it. The instructions say "Edit", so we try that first.

   `bash kubectl edit pvc db-pvc`

3. **Modify Access Mode:** Change:

   `yaml spec: accessModes: - ReadWriteOnce`

   To:

   `yaml spec: accessModes: - ReadWriteMany`

   *Save and exit (`:wq`).*

   *Note: If the API server rejects the update (saying the field is immutable), you must export the YAML, delete the PVC, and recreate it.*

   ```bash

   ## If edit fails:

   kubectl get pvc db-pvc -o yaml > db-pvc.yaml vi db-pvc.yaml # fix the accessMode and remove status/metadata fields kubectl delete pvc db-pvc kubectl apply -f db-pvc.yaml ```

4. **Validation:** Check the status. `bash kubectl get pvc db-pvc` *It should now show `Bound` (assuming the StorageClass can provision it).*

---

## Question 8: Job Failing to Complete
   ```

- **Scenario:** A Job named `data-processor` continuously creates new pods that fail, and the Job never completes.
- **Initial State:** The pod template within the `data-processor` Job specifies a container image `my-app:latest` that does not exist in the registry. The pods fail with an `ImagePullBackOff` error.
- **Task:** Inspect the logs of one of the failed pods created by the Job to identify the image pull error. Edit the `data-processor` Job and correct the container image name to a valid one, such as `busybox`.
- **Validation:** Delete the failed pods. The Job should create a new pod that runs to completion. Verify by running `kubectl get jobs data-processor` and checking that `COMPLETIONS` is `1/1`.

## Solution

Kubernetes Jobs handle batch processes. If the pods created by a Job fail (e.g., due to configuration errors or application crashes), the Job controller will create new pods (up to `backoffLimit`) to try to complete the task. In this scenario, an invalid image causes an infinite failure loop until the backoff limit is reached.

**Step-by-Step Solution:**

1. **Identify the Failed Pods:** List the pods to see the status.

   `bash kubectl get pods`

   *You will see pods named `data-processor-xxxxx` with status `ImagePullBackOff` or `ErrImagePull`.*

2. **Investigate the Error:** Describe a pod to verify the cause.

   `bash kubectl describe pod <pod-name>`

   *Check the Events section for "Failed to pull image".*

3. **Edit the Job:** You cannot change the `template` of an existing Job mostly. However, for the exam/practice, you might be able to edit it if the field is mutable, or more likely, you will need to replace it. *Try editing first:*

   `bash kubectl edit job data-processor`

*Change `image: my-app:latest` to `image: busybox`. If you receive an error stating the field is immutable, proceed to the next method.*

*Method 2: Replace (safest)*

```bash kubectl get job data-processor -o yaml > job.yaml vi job.yaml

## Change image to 'busybox'

## Remove 'controller-uid' and 'resourceVersion' from metadata

## Remove 'status' section

kubectl replace --force -f job.yaml ```

*(Note: `replace --force` deletes and recreates it).*

4. **Clean Up (If not using replace):** If you successfully edited the job or if previous failed pods are still there, you might want to delete the failed pods to force a new one immediately, though the Job controller handles this.

   `bash kubectl delete pods -l job-name=data-processor`

5. **Validation:** Watch the job status. `bash kubectl get jobs data-processor -w` *It should eventually show Completions `1/1`.*

---

## Question 9: Misconfigured Network Policy Blocking Traffic

- **Scenario:** A web application in the `web` namespace cannot connect to its database in the `db` namespace.
- **Initial State:** A default-deny ingress Network Policy is applied to the `db` namespace. An additional Network Policy exists that is intended to allow traffic from the `web` namespace, but its `namespaceSelector` is misconfigured.
- **Task:** Examine the Network Policies in the `db` namespace. Identify the policy intended to allow ingress from `web` and correct its `namespaceSelector` to properly match the labels of the `web` namespace.
```

- **Validation:** Exec into a pod in the `web` namespace and successfully connect to the database service in the `db` namespace using a tool like `curl` or `netcat`.

## Solution

Network Policies use labels to select pods and namespaces. A common mistake is using `matchLabels` on a `namespaceSelector` that doesn't match the actual labels on the namespace object itself (which are different from pod labels).

**Step-by-Step Solution:**

1. **Check Namespace Labels:** First, find out what labels the `web` namespace actually has.

   `bash kubectl get namespace web --show-labels`

   *Note the labels, e.g., `name=web` or `project=web`.*

2. **Inspect Policies:** List the policies in the `db` namespace.

   `bash kubectl get netpol -n db`

   *Identify the one that looks like it should allow access (e.g., `allow-web`).*

3. **Check Policy Configuration:** Describe the policy to see its selectors.

   `bash kubectl describe netpol allow-web -n db`

   *Look at the `Ingress` rules. You might see `NamespaceSelector: project=web` but the namespace actually has `name=web`.*

4. **Edit the Policy:** Correct the selector.

   `bash kubectl edit netpol allow-web -n db`

   *Change the `namespaceSelector` to match the actual label found in Step 1.*

   `yaml ingress: - from: - namespaceSelector: matchLabels: name: web # Ensure this matches the output from Step 1`

   *Save and exit (`:wq`).*

5. **Validation:** Test connectivity. `bash # Assuming there is a pod 'web-pod' in 'web' and a service 'db-svc' in 'db' kubectl exec -n web web-pod -- curl -m 2 db-svc.db`

`# OR using nc kubectl exec -n web web-pod -- nc -z -v -w 2 db-svc.db 5432` *The command should succeed.*

---

## Question 10: Scheduler Failure

- **Scenario:** Newly created pods are remaining in the `Pending` state indefinitely. The cluster scheduler appears to be non-functional.
- **Initial State:** The `kube-scheduler` static pod manifest at `/etc/kubernetes/manifests/kube-scheduler.yaml` on the control plane node references a non-existent configuration file via the `--config` flag.
- **Task:** SSH to the control plane node. Inspect the `kube-scheduler` logs using `crictl logs` (or equivalent) to find the error about the missing config file. Edit the manifest at `/etc/kubernetes/manifests/kube-scheduler.yaml` and remove the invalid `--config` flag to allow the scheduler to start with its default configuration.
- **Validation:** Create a new NGINX pod. It should be scheduled and transition to the Running state.

### Solution

The kube-scheduler is responsible for assigning pods to nodes. If it fails, pods sit in the `Pending` state forever. In a kubeadm cluster, the scheduler runs as a static pod, meaning its definition lives in `/etc/kubernetes/manifests`. Debugging involves checking why the container keeps exiting.

**Step-by-Step Solution:**

1. **Confirm Scheduler Issue:** Check the system components.

   `bash kubectl get pods -n kube-system`

   *You will likely see `kube-scheduler-controlplane` in `CrashLoopBackOff` or seemingly missing if it's failing fast.*

2. **SSH to Control Plane:** Access the node.

   `bash ssh control-plane`

3. **Inspect Logs:** Since the pod is crashing, `kubectl logs` might work, but `crictl` is more reliable on the node.

```bash crictl ps -a | grep scheduler

## Get the container ID from the output

crictl logs ```

*Look for errors like:* `stat /etc/kubernetes/scheduler.conf: no such file or directory`.

4. **Edit the Manifest:** Modify the static pod definition.

   `bash cd /etc/kubernetes/manifests vi kube-scheduler.yaml`

   *Find the line containing the invalid flag, e.g.,* `--config=/etc/kubernetes/scheduler.conf`. *Delete this line or correct the path if you knew the correct one. In this scenario, we remove it to revert to defaults.*

5. **Save and Exit:** Save the file (`:wq`). The Kubelet will automatically restart the static pod.

6. **Verify Recovery:** Wait a few seconds.

   ```bash crictl ps | grep scheduler

   ## Should show Running

   ```

   *Exit SSH.*

7. **Validation:** Create a test pod. `bash kubectl run test-schedule --image=nginx kubectl get pod test-schedule -w` *It should move from Pending to ContainerCreating to Running.*

---

## Domain: Cluster Architecture, Installation & Configuration (10 Questions)

## Question 11: Install a Component with Helm
   ```

- **Scenario:** The monitoring team requires Prometheus to be installed in the cluster to collect metrics.
- **Initial State:** A namespace `monitoring` exists. Helm is installed.
- **Task:** Add the `prometheus-community` Helm repository. Install the Prometheus chart from this repository into the `monitoring` namespace, giving it the release name `prom-stack`.
- **Validation:** Run `helm list -n monitoring`. It should show the `prom-stack` release in a deployed status.

## Solution

Helm is the package manager for Kubernetes. It uses "Charts" (packages) stored in "Repositories". To install a chart, you first add the repository, update your local cache, and then run the install command.

**Step-by-Step Solution:**

1. **Add the Repository:** Add the `prometheus-community` repo to your Helm configuration.

   `bash helm repo add prometheus-community https://prometheus-community.github.io/helm-charts`

2. **Update Repo Cache:** Ensure you have the latest chart versions.

   `bash helm repo update`

3. **Search for the Chart (Optional):** Verify the chart exists and find the correct name.

   `bash helm search repo prometheus-community/prometheus`

4. **Install the Chart:** Install `prometheus` with the release name `prom-stack` into the `monitoring` namespace.

   `bash helm install prom-stack prometheus-community/prometheus --namespace monitoring`

   *If the namespace does not exist (though the scenario says it does), you would add `--create-namespace`.*

5. **Validation:** Check the installed release.

   `bash helm list -n monitoring`

*You should see `prom-stack` with status `deployed`.*

To fix the permission error, we need to instruct the Prometheus server to run with permissions that allow it to write to the storage volume. Since you are running in Minikube, the most reliable fix is to run the container as the root user.

```
helm upgrade prom-stack prometheus-community/prometheus --namespace monitoring \
  --set server.securityContext.runAsUser=0 \
  --set server.securityContext.runAsNonRoot=false \
  --set server.securityContext.runAsGroup=0 \
  --set server.securityContext.fsGroup=0

kubectl get pods -n monitoring -w
```

---

## Question 12: Helm Install with Custom Values

- **Scenario:** An existing Helm installation of cert-manager needs to be configured to not install its CRDs, as they will be managed manually.
- **Initial State:** The `jetstack` Helm repository has been added.
- **Task:** Install the `cert-manager` chart from the `jetstack` repository into the `cert-manager` namespace. During installation, use the `--set` flag to set the `installCRDs` value to `false`.
- **Validation:** Run `kubectl get crds | grep cert-manager`. No CRDs related to cert-manager should be present.

### Solution

Helm charts allow configuration via values. `cert-manager` extensively uses Custom Resource Definitions (CRDs) to manage certificates. By default, the chart might try to install them, but in production environments, it is often recommended to manage CRDs separately (or they might already exist) to avoid accidental deletion during chart uninstallation.

**Step-by-Step Solution:**

1. **Add/Update the Repository:** Ensure the `jetstack` repository is added and updated.

   `bash helm repo add jetstack https://charts.jetstack.io helm repo update`

2. **Create Namespace:** Create the `cert-manager` namespace if it doesn't restrict.

   `bash kubectl create namespace cert-manager --dry-run=client -o yaml | kubectl apply -f -`

*This command generates the namespace definition in YAML format without sending it to the API server (`--dry-run=client -o yaml`) and pipes it to `kubectl apply`. This provides an idempotent way to ensure the namespace exists without failing if it was already created.*

3. **Install with Custom Flag:** Install `cert-manager` setting `installCRDs` to `false`.

   `bash helm install cert-manager jetstack/cert-manager \ --namespace cert-manager \ --set installCRDs=false`

4. **Validation:** Verify that no CRDs were created by the helm chart.

   `bash kubectl get crds | grep cert-manager`

   *This command should return no results.*

   *Check the release status:*

   `bash helm list -n cert-manager`

---

## Question 13: List and Explain a CRD

- **Scenario:** An operator for Vault has been installed, and you need to document one of its custom resources.
- **Initial State:** The cluster contains several CRDs, including `vaultsecrets.secrets.hashicorp.com`.
- **Task:** List all CRDs in the cluster and save the list to a file named `/opt/crds.txt`. Then, use `kubectl explain` to find the documentation for the `spec.vault.address` field of the `VaultSecret` CRD and save this documentation to `/opt/crd_field.txt`.
- **Validation:** Check the contents of the two files, `/opt/crds.txt` and `/opt/crd_field.txt`, to ensure they contain the correct information.

### Solution

Custom Resource Definitions (CRDs) extend the Kubernetes API. Once installed, `kubectl` treats them like native resources. You can list them using `kubectl get crds` and explore their

schema/documentation using `kubectl explain`, just like you would for a Pod or Service.

**Step-by-Step Solution:**

1. **List CRDs:** Get all custom resource definitions and save them to the specified file.

   `bash kubectl get crds > /opt/crds.txt`

   *You can verify the list contains the Vault CRD:*

   `bash grep vault /opt/crds.txt`

2. **Explain the Field:** Use `kubectl explain` to drill down into the schema of the `VaultSecret` resource. The question asks for `spec.vault.address`.

   ```bash

   ## Syntax: kubectl explain .

   kubectl explain VaultSecret.spec.vault.address > /opt/crd_field.txt ```

   *Note: If `VaultSecret` (Kind) doesn't work, try the plural name or full name if necessary, e.g., `vaultsecrets.spec.vault.address`.*

3. **Validation:** Verify the files were created and contain content.

   `bash cat /opt/crds.txt cat /opt/crd_field.txt`

---

## Question 14: Create a Specific RBAC Role

- **Scenario:** A new developer, `david`, needs read-only access to pods and services in the `dev` namespace.
- **Initial State:** The `dev` namespace exists. A user `david` is present in the cluster's authentication system.
- **Task:** Create a Role named `pod-service-reader` in the `dev` namespace that grants `get`, `list`, and `watch` permissions on pods and services. Then, create a RoleBinding named `david-reader-binding` to bind the `david` user to this new role.
- **Validation:** Run `kubectl auth can-i get pods --as david -n dev`. The result should be `yes`. Run `kubectl auth can-i delete pods --as david -n dev`. The result should be
   ```

`no`.

### Solution

RBAC (Role-Based Access Control) in Kubernetes is used to regulate access to computer or network resources.

- **Role**: Defines permissions (verbs like `get`, `list`) on resources (like `pods`) within a specific namespace.
- **RoleBinding**: Grants the permissions defined in a Role to a User, Group, or ServiceAccount.

**Step-by-Step Solution:**

1. **Create the Role:** Use the imperative command to create the role `pod-service-reader` in namespace `dev` with the specified verbs and resources.

   `bash kubectl create role pod-service-reader --verb=get,list,watch --resource=pods,services -n dev`

   *Alternatively, define it via YAML:*

   `yaml apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: namespace: dev name: pod-service-reader rules: - apiGroups: [""] # "" indicates the core API group resources: ["pods", "services"] verbs: ["get", "list", "watch"]`

2. **Create the RoleBinding:** Bind the user `david` to the role `pod-service-reader`.

   `bash kubectl create rolebinding david-reader-binding --role=pod-service-reader --user=david -n dev`

   *YAML Alternative:*

   `yaml apiVersion: rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: david-reader-binding namespace: dev subjects: - kind: User name: david # Name is case sensitive apiGroup: rbac.authorization.k8s.io roleRef: kind: Role name: pod-service-reader apiGroup: rbac.authorization.k8s.io`

3. **Validation:** Use the `auth can-i` command to verify permissions as the user `david`.

   ```bash

## Should return 'yes'
   ```

kubectl auth can-i get pods --as david -n dev

## Should return 'no'

kubectl auth can-i delete pods --as david -n dev ```

---

## Question 15: Create a Cluster-Wide RBAC Role

- **Scenario:** A cluster-wide auditing tool needs permission to view all nodes and persistent volumes in the cluster.
- **Initial State:** A `ServiceAccount` named `auditor` exists in the `kube-system` namespace.
- **Task:** Create a `ClusterRole` named `node-pv-viewer` that grants `get` and `list` permissions on nodes and persistent volumes. Create a `ClusterRoleBinding` named `auditor-global-binding` to grant the `auditor` ServiceAccount this ClusterRole.
- **Validation:** Run `kubectl auth can-i list nodes --as=system:serviceaccount:kube-system:auditor`. The result should be `yes`.

### Solution

While `Roles` and `RoleBindings` are namespaced, `ClusterRoles` and `ClusterRoleBindings` are cluster-wide. This is required for:

1. Cluster-scoped resources (like Nodes, PersistentVolumes).
2. Granting access multiple namespaces or the entire cluster.

**0. Setup Initial State:** Ensure the ServiceAccount exists as per the scenario.

```
kubectl create serviceaccount auditor -n kube-system
```

**Step-by-Step Solution:**

1. **Create ClusterRole:** Define the permissions for cluster-scoped resources.

   `bash kubectl create clusterrole node-pv-viewer --verb=get,list --resource=nodes,persistentvolumes`

*Alternatively via YAML:*

`yaml apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRole metadata: name: node-pv-viewer rules: - apiGroups: [""] resources: ["nodes", "persistentvolumes"] verbs: ["get", "list"]`

2. **Create ClusterRoleBinding:** Bind the ServiceAccount to the ClusterRole. Note that for ServiceAccounts, you must specify the namespace they belong to.

`bash kubectl create clusterrolebinding auditor-global-binding \ --clusterrole=node-pv-viewer \ --serviceaccount=kube-system:auditor`

*Alternatively via YAML:*

`yaml apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRoleBinding metadata: name: auditor-global-binding subjects: - kind: ServiceAccount name: auditor namespace: kube-system roleRef: kind: ClusterRole name: node-pv-viewer apiGroup: rbac.authorization.k8s.io`

3. **Validation:** Check permissions impersonating the ServiceAccount username. The username format for a ServiceAccount is `system:serviceaccount:<namespace>:<name>`.

```bash kubectl auth can-i list nodes --as=system:serviceaccount:kube-system:auditor

## Expected: yes

```

---

## Question 16: Upgrade a kubeadm Cluster

- **Scenario:** The cluster is running Kubernetes version `1.28.1` and needs to be upgraded to the latest patch release of `1.28`.
- **Initial State:** A single-node kubeadm cluster is running version `1.28.1`.
- **Task:** On the control plane node, update the package manager and install the target version of `kubeadm`. Run `kubeadm upgrade plan` to verify the upgrade path. Then, apply the upgrade using `kubeadm upgrade apply v1.28.x` (where `x` is the latest patch). After the control plane is upgraded, drain the node, upgrade `kubelet` and `kubectl`, and then uncordon the node.

- **Validation:** Run `kubectl get nodes`. The version column should reflect the new Kubernetes version.

### Solution

Upgrading a Kubeadm cluster involves a specific order of operations:

1. **Upgrade kubeadm** on the node.
2. **Drain** the node (to prepare for downtime/restarts).
3. **Upgrade the cluster** using `kubeadm upgrade apply` (for control plane) or `node` (for workers).
4. **Upgrade kubelet and kubectl** on the node.
5. **Restart kubelet** and **Uncordon** the node.

**Step-by-Step Solution:**

*Note: The commands below assume a Debian/Ubuntu-based system, which is standard for the CKA exam.*

1. **Update Package Repository:** Update the `apt` cache to find new versions.

   `bash apt-get update`

2. **Find Available Versions:** Check specifically for the 1.28 versions.

   ```bash apt-cache madison kubeadm | grep 1.28

   ## Example Output:

   ---

   ## kubeadm | 1.28.2-1.1 | https://pkgs.k8s.io/core:/stable:/v1.28/deb/ Packages

   ---

   ```

   *Assume we identify `1.28.2-1.1` as the target.*

3. **Upgrade kubeadm:** Install/Upgrade the `kubeadm` binary first. To prevent it from being upgraded automatically in the future, we use `install` with specific version, and then `apt-`

`mark hold` is a good practice (often already held).

`bash apt-mark unhold kubeadm apt-get install -y kubeadm=1.28.2-1.1 apt-mark hold kubeadm`

*Verify version:* `kubeadm version`

4. **Plan the Upgrade:** Verify the upgrade plan looks correct.

   `bash kubeadm upgrade plan`

   *This checks pre-flight requirements and shows you the versions you can upgrade to.*

5. **Apply the Upgrade:** This upgrades the control plane components (etcd, API server, controller-manager, scheduler).

   `bash sudo kubeadm upgrade apply v1.28.2`

   *Wait for the "SUCCESS" message.*

6. **Drain the Node:** Prepare the node for kubelet upgrade.

   `bash kubectl drain <node-name> --ignore-daemonsets`

   *(If running on the same node, use `localhost` or the node name found in `kubectl get nodes`).*

7. **Upgrade kubelet and kubectl:** Upgrade the remaining binaries.

   `bash apt-mark unhold kubelet kubectl apt-get install -y kubelet=1.28.2-1.1 kubectl=1.28.2-1.1 apt-mark hold kubelet kubectl`

8. **Restart Kubelet:** Reload the configuration.

   `bash systemctl daemon-reload systemctl restart kubelet`

9. **Uncordon the Node:** Bring the node back into scheduling.

   `bash kubectl uncordon <node-name>`

10. **Validation:** Check the node status and version.

    `bash kubectl get nodes`

    *The `VERSION` column should now show `v1.28.2` and status `Ready`.*

---

## Question 17: Backup etcd

- **Scenario:** As part of a disaster recovery plan, you need to perform a manual backup of the etcd database.
- **Initial State:** The cluster is running with an etcd instance managed by kubeadm as a static pod.
- **Task:** Using the `etcdctl` binary, create a snapshot of the etcd database. You will need to provide the correct endpoint, CA certificate, client certificate, and key, which can be found by inspecting the etcd static pod manifest. Save the snapshot to `/opt/etcd-backup.db`.
- **Validation:** Run `etcdctl snapshot status /opt/etcd-backup.db` to verify the integrity of the backup file.

### Solution

All cluster data is stored in **etcd**. Backing it up is critical for restoring the cluster in case of a crash. Since etcd is secured with mutual TLS (mTLS), we must provide the certificates to the `etcdctl` tool to authorize the snapshot backup.

**Step-by-Step Solution:**

1. **Retrieve Certificate Paths:** The etcd server configuration is defined in its static pod manifest. We need to find the paths for:
   - `--trusted-ca-file` (CACert)
   - `--cert-file` (Cert)
   - `--key-file` (Key)
   - `--listen-client-urls` (Endpoint)

   ```bash

   # Inspect the manifest

   cat /etc/kubernetes/manifests/etcd.yaml | grep file ```

   *Typical output locations (standard kubeadm):*

   - Point 1 (CACert): `/etc/kubernetes/pki/etcd/ca.crt`
   - Point 2 (Cert): `/etc/kubernetes/pki/etcd/server.crt`
   ```

- Point 3 (Key): `/etc/kubernetes/pki/etcd/server.key`

2. **Take the Snapshot:** Run the `etcdctl` command. Ensure `ETCDCTL_API=3` is set (it is the default in newer versions, but good practice to be explicit).

```
bash ETCDCTL_API=3 etcdctl \ --endpoints=https://127.0.0.1:2379 \ --cacert=/etc/kubernetes/pki/etcd/ca.crt \ --cert=/etc/kubernetes/pki/etcd/server.crt \ --key=/etc/kubernetes/pki/etcd/server.key \ snapshot save /opt/etcd-backup.db
```

3. **Validation:** Verify the snapshot file was created and is valid.

```
bash ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/etcd-backup.db
```

*This should display a table with Hash, Revision, and Total Key details.*

---

## Question 18: Add a New Worker Node

- **Scenario:** The cluster needs more capacity, and a new machine is ready to be joined as a worker node.
- **Initial State:** A one-control-plane, one-worker cluster exists. A new machine `new-worker` is provisioned with a container runtime.
- **Task:** On the control plane node, generate a new `kubeadm join` token. SSH to the `new-worker` machine and use the generated command (`kubeadm join ...`) to join the node to the cluster.
- **Validation:** From the control plane, run `kubectl get nodes`. The `new-worker` node should appear in the list with a `Ready` status after a few moments.

### Solution

Bootstrapping a new node into a Kubernetes cluster involves trust establishment. The `kubeadm join` command uses a token (shared secret) to authenticate with the control plane and a discovery token hash to validate the control plane's identity.

**Step-by-Step Solution:**

1. **Generate Join Command:** Run this on the **control plane** node. This command generates a valid token and prints the full `kubeadm join` command needed by the worker.

```
bash kubeadm token create --print-join-command
```

*Output will look like:* `kubeadm join <control-plane-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>`

2. **Access the Worker Node:** SSH into the new machine.

```
bash ssh new-worker
```

3. **Run the Join Command:** Paste the command generated in Step 1. You needs `sudo` or root privileges.

```
bash sudo kubeadm join <control-plane-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

*Wait for the "This node has joined the cluster" message.*

4. **Validation:** Go back to the control plane and check the node list.

```
bash exit # Exit ssh session kubectl get nodes
```

*The new node should be in `Ready` state (or `NotReady` initially until the CNI plugin initializes pod networking on it).*

---

## Question 19: Use Kustomize to Apply a Variant

- **Scenario:** An application has base configurations, but you need to deploy a staging variant with increased replica counts.
- **Initial State:** A directory `/opt/app` contains a `kustomization.yaml` and a `deployment.yaml` (with 1 replica). A subdirectory `/opt/app/overlays/staging` contains another `kustomization.yaml`.
- **Task:** Edit the Kustomization file in `/opt/app/overlays/staging` to use the base configuration but patch the Deployment to have 3 replicas. Then, apply the staging configuration to the cluster using `kubectl apply -k /opt/app/overlays/staging`.
- **Validation:** Run `kubectl get deployment` and verify that the application's deployment has 3 replicas running.

### Solution

Kustomize is a configuration management tool built into `kubectl`. It allows you to define a "base" set of resources and then create "overlays" (variants) that patch the base for different environments (e.g., dev, staging, prod) without duplicating the entire YAML.

- **Base:** The common configuration.
- **Overlay:** Specific changes (like `replicas` or `resource` limits) applied on top of the base.

**Step-by-Step Solution:**

1. **Navigate to Overlay Directory:** Go to where the staging configuration is defined.

   `bash cd /opt/app/overlays/staging`

2. **Edit Kustomization File:** You need to tell Kustomize to patch the `replicas` field. While you can use a separate patch file, Kustomize also supports inline patches or specific transformers like `replicas`.

   *Method A: Using the `replicas` field (Simplest)* Open `kustomization.yaml`:

   `bash vi kustomization.yaml`

   Add/Modify:

   ```yaml resources: - ../../base # Points to the base directory

   replicas: - name: my-app-deployment # Name of the deployment in the base count: 3 ```

   *Method B: Using an inline patch*

   ```yaml resources: - ../../base

   patches: - target: group: apps version: v1 kind: Deployment name: my-app-deployment patch: |- op: replace path: /spec/replicas value: 3 ```

   *(Method A is preferred for simple replica changes).*

3. **Apply with Kustomize:** Use `kubectl apply -k <directory>`.

   ```bash kubectl apply -k .

## OR
   ```

kubectl apply -k /opt/app/overlays/staging ```

4. **Validation:** Check that the deployment was updated (or created) with 3 replicas.

   `bash kubectl get deployment`

   *Ensure the `READY` column says `3/3`.*

---

## Question 20: Configure an External CRI

- **Scenario:** A node is configured to use dockerd, but policy requires it to use containerd.
- **Initial State:** A worker node `worker-1` has both dockerd and containerd installed. The kubelet is configured to use the docker runtime socket.
- **Task:** SSH to `worker-1`. Drain the node. Stop the kubelet. Edit the kubelet configuration (`/var/lib/kubelet/kubeadm-flags.env` or similar) to point to the containerd socket (`--container-runtime-endpoint=unix:///run/containerd/containerd.sock`). Restart the kubelet. Uncordon the node.
- **Validation:** Run `kubectl describe node worker-1`. The Container Runtime Version should show `containerd://...`.

### Solution

The Container Runtime Interface (CRI) allows Kubernetes to support various container runtimes. Changing the runtime involves ensuring the new runtime is installed/active and then reconfiguring the `kubelet` to point to the new runtime's socket.

- **Docker Shim (Legacy):** `unix:///var/run/dockershim.sock` (Deprecated/Removed in 1.24+)
- **Containerd:** `unix:///run/containerd/containerd.sock`

**Step-by-Step Solution:**

1. **Drain the Node:** Schedule downtime for the node.

   `bash kubectl drain worker-1 --ignore-daemonsets`

2. **SSH to the Node:** Access the worker node.

   `bash ssh worker-1`

3. **Stop Kubelet:** Stop the service before making changes.

   `bash systemctl stop kubelet`

4. **Edit Configuration:** Detailed configuration for kubeadm-managed nodes is often passed via environment variables in `/var/lib/kubelet/kubeadm-flags.env`. This file constructs the flags passed to the kubelet binary.

   `bash vi /var/lib/kubelet/kubeadm-flags.env`

   *Find the `--container-runtime-endpoint` flag (or add it if missing) and set it to: `--container-runtime-endpoint=unix:///run/containerd/containerd.sock`*

   *Example line:* `KUBELET_KUBEADM_ARGS="--container-runtime-endpoint=unix:///run/containerd/containerd.sock --pod-infra-container-image=..."`

5. **Restart Kubelet:** Start the service.

   `bash systemctl start kubelet`

6. **Uncordon Usage:** Return to the control plane (exit SSH) and enable the node.

   `bash exit kubectl uncordon worker-1`

7. **Validation:** Check the CRI version.

   `bash kubectl describe node worker-1 | grep "Container Runtime"`

   *It should show `containerd://<version>` instead of `docker://...`.*

---

## Domain: Services & Networking (10 Questions)

## Question 21: Expose a Deployment with a NodePort Service

- **Scenario:** A deployment named `webapp` is running in the `app-space` namespace, and it needs to be accessible from outside the cluster for testing purposes.
- **Initial State:** A Deployment `webapp` with 2 replicas is running in the `app-space` namespace. The pods expose port `8080`.

- **Task:** Create a Service named `webapp-svc` of type `NodePort` in the `app-space` namespace. The service should expose port `80` on the service itself, target port `8080` on the pods, and expose node port `30080` on the cluster nodes.
- **Validation:** Run `curl <node-ip>:30080` from the base node. It should return a response from the `webapp`.

## Solution

To expose the application externally on a specific port on every node, we use a specific `nodePort` in the Service definition. The `kubectl expose` command is useful to generate the template with the correct selectors, but the `nodePort` field usually must be added manually in the YAML or by editing after generation.

**Imperative Command to Generate YAML:**

```
# Generate the file with correct selectors automatically
kubectl expose deployment webapp -n app-space --name=webapp-svc --type=NodePort --port=

# Manually edit webapp-svc.yaml to include 'nodePort: 30080' in the ports section
# Then apply
kubectl apply -f webapp-svc.yaml
```

**YAML Manifest:**

```
apiVersion: v1
kind: Service
metadata:
  name: webapp-svc
  namespace: app-space
spec:
  selector:
    app: webapp # Note: Ensure this matches the Deployment's labels (check with: kubect
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
```

---

## Question 22: Create an Ingress Resource

- **Scenario:** An application, served by `app-svc`, needs to be exposed externally via HTTP routing at the path `/app`.
- **Initial State:** An Ingress controller is running. A service `app-svc` exists in the `default` namespace and exposes port `80`.
- **Task:** Create an Ingress resource named `app-ingress`. It should route traffic for the path `/app` to the `app-svc` service on port `80`.
- **Validation:** Find the external IP of the Ingress controller and run `curl http://<ingress-ip>/app`. It should connect to the `app-svc`.

### Solution & Minikube Practice Steps

To practice this in Minikube, you must first ensure the ingress controller is enabled.

**1. Setup Environment:**

```
# Enable the NGINX Ingress controller
minikube addons enable ingress

# Create a sample application and service to act as the backend
kubectl create deployment web --image=nginx
kubectl expose deployment web --name=app-svc --port=80
```

**2. Create the Ingress (The Solution):** You can create the ingress imperatively or using a YAML manifest.

*Imperative Command:*

```
kubectl create ingress app-ingress --rule="/app=app-svc:80" --class=nginx
```

*Note: The `--class=nginx` flag is important in modern clusters./*

*YAML Manifest:*

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
  name: app-ingress
  annotations:
    # Optional: Necessary if the backend app expects root path '/' but receives '/app'
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /app
            pathType: Prefix
            backend:
              service:
                name: app-svc
                port:
                  number: 80
```

**3. Troubleshooting & Validation:**

If you receive a **404 Not Found** error when accessing `/app`, it is because the backend application (Nginx) expects requests at the root path `/` but is receiving `/app`. You need to add a rewrite annotation.

**Fix 404 Error:**

```
# Add the rewrite annotation to existing ingress
kubectl annotate ingress app-ingress nginx.ingress.kubernetes.io/rewrite-target=/
```

**Accessing via Localhost (Minikube Tunnel):** If `minikube ip` is not accessible or you are on a driver that requires tunneling (like Docker on Mac):

```
# Open a tunnel in a separate terminal
sudo minikube tunnel -p <profile_name> # e.g., minikube or cka-multinode

# Test via localhost
curl -L http://127.0.0.1/app
```

**Standard Validation:**

```
# Check the ingress status and IP
kubectl get ingress app-ingress
```

```
# Test the access
curl http://$(minikube ip)/app
```

---

## Question 23: Ingress to Gateway API Migration

- **Scenario:** The organization is standardizing on the Gateway API. An existing application exposed via Ingress must be migrated.
- **Initial State:** An Ingress resource `legacy-ingress` routes traffic for `app.example.com` to `app-svc`. A `GatewayClass` named `gw-class` exists. A TLS secret `app-tls` exists.
- **Task:** Create a `Gateway` resource named `main-gateway` that uses the `gw-class` and listens on port `443` for HTTPS traffic from `app.example.com`, using the `app-tls` secret. Create an `HTTPRoute` named `app-route` that attaches to this gateway and routes requests for `app.example.com` to the `app-svc` service. After verifying the route works, delete the `legacy-ingress`.
- **Validation:** Test connectivity to `https://app.example.com` (using `curl --resolve`). After confirming it works, verify that `kubectl get ingress legacy-ingress` returns “not found.”

### Solution & Concept Explanation

#### Concept: Why Gateway API?

Gateway API is the successor to Ingress.

- **Role-Oriented:** It splits responsibilities.
- **Infrastructure Provider** manages `GatewayClass` (defines the controller, e.g., NGINX vs Istio).
- **Cluster Operator** manages `Gateway` (defines entry points, ports, and TLS termination).
- **Developer** manages `HTTPRoute` (defines URI routing to Services).
- **Expressiveness:** Better support for header matching, traffic splitting (canary), and cross-namespace routing than standard Ingress.
- **Extensibility:** Designed to support more than just HTTP (e.g., GRPC, TCP).

#### Concept: TLS Secrets

A Secret of type `kubernetes.io/tls` stores a certificate (`tls.crt`) and a private key (`tls.key`). In Gateway API, the `Gateway` resource references this Secret in its `listeners` section to terminate SSL/TLS traffic before forwarding it to the backend services.

**Minikube Lab Setup**

Gateway API is not installed by default. You must install the CRDs and a Controller (like NGINX Gateway Fabric).

**1. Install Gateway API CRDs & Controller (NGINX Gateway Fabric):**

```
# Install Standard Gateway API CRDs
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.

# Install NGINX Gateway Fabric CRDs
kubectl apply -f https://raw.githubusercontent.com/nginx/nginx-gateway-fabric/v1.3.0/de
# OR (if using newer version setup):
# kubectl apply -k "https://github.com/nginx/nginx-gateway-fabric/config/default?ref=v1
```

**2. Setup Initial State (Legacy Ingress):**

```
# Create Backend
kubectl create deployment app-svc --image=nginx --port=80
kubectl expose deployment app-svc --port=80

# Create TLS Secret (Self-Signed)
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out tls.crt -subj
kubectl create secret tls app-tls --key tls.key --cert tls.crt

# Create Legacy Ingress
kubectl create ingress legacy-ingress --rule="app.example.com/=app-svc:80,tls=app-tls"
```

**3. The Solution (Migration):**

**Step A: Create GatewayClass (Already exists if using NGINX Fabric, named 'nginx')**

**Step B: Create Gateway (Listening on 443 with TLS)**

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
```

```
  name: main-gateway
spec:
  gatewayClassName: nginx
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      hostname: app.example.com
      tls:
        mode: Terminate
        certificateRefs:
          - name: app-tls
```

### Step C: Create HTTPRoute (Routing Logic)

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
    - name: main-gateway
  hostnames:
    - "app.example.com"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: app-svc
          port: 80
```

### 4. Validation:

```
# Get IP of Gateway (via Tunnel if needed)
kubectl get gateway main-gateway

# Test connectivity
curl -k --resolve app.example.com:443:127.0.0.1 https://app.example.com
```

```
# Cleanup Legacy
kubectl delete ingress legacy-ingress
```

---

## Question 24: Restrict Ingress Traffic with a Network Policy

- **Scenario:** A database in the `db` namespace should only accept connections from the application frontend in the `frontend` namespace.
- **Initial State:** Pods in the `frontend` namespace (with label `app=frontend`) need to connect to pods in the `db` namespace (with label `app=db`) on port `5432`.
- **Task:** Create a Network Policy named `db-allow-frontend` in the `db` namespace. This policy should apply to pods with the label `app=db`. It should define an ingress rule that allows traffic on TCP port `5432` only from pods in a namespace that has the label `name=frontend`.
- **Validation:** Exec into a pod in the `frontend` namespace and confirm it can connect to the DB service. Exec into a pod in another namespace (e.g., `default`) and confirm the connection is blocked.

### Full Solution & Explanation

#### Concept: Cross-Namespace Access

By default, standard `podSelector` rules only match pods **within the same namespace** as the NetworkPolicy. To allow traffic from a *different* namespace, you must use `namespaceSelector`.

- **`namespaceSelector`**: Matches the **labels of the Namespace object** (not the pods inside it).
- **`podSelector`**: Matches the **labels of the Pods** inside the selected namespace.

#### Step 1: Environment Setup (Replicate the Scenario)

First, we need to create the namespaces and pods to simulate the problem.

```
# 1. Create Namespaces
kubectl create namespace db
kubectl create namespace frontend

# 2. LABEL THE FRONTEND NAMESPACE (Crucial Step!)
# The NetworkPolicy will look for this label on the NAMESPACE itself.
kubectl label namespace frontend name=frontend
```

```
# 3. Create the Database Pod
kubectl run db --image=redis --labels="app=db" -n db

# 4. Create the Frontend Pod
kubectl run frontend --image=nginx --labels="app=frontend" -n frontend
```

### Step 2: The Solution (Network Policy)

Create the policy in the `db` namespace to protect the `db` pod.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-frontend
  namespace: db
spec:
  podSelector:
    matchLabels:
      app: db # Selects the DB pod in 'db' namespace
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              name: frontend # Matches the namespace label we added in Step 1
          # Note: We do NOT add a podSelector here, so ANY pod in the 'frontend' namesp
      ports:
        - protocol: TCP
          port: 6379 # Redis port (or 5432 if using Postgres)
```

*(Note: I used Redis (port 6379) for the example pod as it's lighter than Postgres, but the concept is identical for port 5432).*

### Step 3: Validation

```
# 1. Test from frontend (Should Work)
kubectl exec -it frontend -n frontend -- timeout 2 curl <db-pod-ip>:6379
# (Or use nc/telnet if installed)
```

```
# 2. Test from another namespace (Should Fail/Timeout)
kubectl run hacker --image=nginx --restart=Never --rm -it -- timeout 2 curl <db-pod-ip>
```

---

## Question 25: Allow Egress Traffic with a Network Policy

- **Scenario:** An application pod needs to make API calls to an external service at `192.0.2.10/32`, but all other outbound traffic should be blocked.
- **Initial State:** A pod with label `app=api-client` in the `default` namespace. A default-deny egress policy is in effect for the namespace.
- **Task:** Create a Network Policy named `allow-external-api` that applies to pods with `app=api-client`. The policy should define an egress rule that allows traffic to the IP block `192.0.2.10/32` on TCP port `443`.
- **Validation:** Exec into the `api-client` pod. Confirm that `curl https://192.0.2.10` works, but `curl https://google.com` times out.

### Solution

#### Concept: Egress Isolation

Network Policies are additive (allow-lists). By default, all traffic is allowed. As soon as you apply a Network Policy that selects a pod and specifies "Egress" in `policyTypes`, **ALL** outbound traffic from that pod is blocked, *except* what is explicitly allowed by the rules. This is why we only need to define the rule for `192.0.2.10`. Everything else is automatically blocked because it's not on the list.

#### YAML Manifest

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-external-api
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: api-client
  policyTypes:
    - Egress
```

```
  egress:
    - to:
        - ipBlock: # Use ipBlock for external CIDR ranges
            cidr: 192.0.2.10/32
      ports:
        - protocol: TCP
          port: 443
```

*Note: If you need DNS resolution (e.g., to resolve google.com even if you block access to it), you would also need to allow UDP port 53 to the cluster DNS.*

---

## Question 26: Create a Headless Service

- **Scenario:** A StatefulSet of database replicas requires a headless service for direct pod discovery.
- **Initial State:** A StatefulSet named `db-statefulset` with 3 replicas exists. The pods have the label `app=db`.
- **Task:** Create a headless Service named `db-svc-headless`. It should target pods with the label `app=db`. To make it headless, set the `clusterIP` field to `None`.
- **Validation:** Run `nslookup db-svc-headless`. It should return the individual IP addresses of the three database pods.

### Solution

A **Headless Service** allows you to communicate directly with Pods by returning their individual IP addresses via DNS, rather than a single ClusterIP. This is often used for stateful applications (like databases) where knowing the identity of each replica is important. You create one by setting `clusterIP: None` in the Service spec.

**Imperative Command:**

There isn't a direct flag to set `clusterIP: None` with `kubectl create service`. You usually generate a standard ClusterIP service content and modify it.

```
kubectl create service clusterip db-svc-headless --tcp=80:80 --dry-run=client -o yaml
```

**YAML Manifest:**

Edit the file to add `clusterIP: None` and ensure the selector is correct.

```
apiVersion: v1
kind: Service
metadata:
  name: db-svc-headless
spec:
  clusterIP: None # Key field for Headless Service
  selector:
    app: db # Must match the Pod labels
  ports:
    - port: 80 # Service port
      targetPort: 80 # Pod port
```

**Apply:**

```
kubectl apply -f headless.yaml
```

**Validation:**

```
# Verify the service has no ClusterIP
kubectl get svc db-svc-headless

# DNS Lookup (should return multiple IPs)
kubectl run tmp-shell --rm -it --image=busybox:1.28 -- nslookup db-svc-headless
```

---

## Question 27: Select the Correct Network Policy Manifest

- **Scenario:** You are given three manifest files: `/opt/np-1.yaml`, `/opt/np-2.yaml`, and `/opt/np-3.yaml`. You must apply the one that implements a specific security rule.
- **Initial State:** Three YAML files containing Network Policy definitions.
- **Task:** Analyze the files to determine which one correctly implements the following rule: “In the backend namespace, allow ingress traffic on port 8000 to pods with label `role=api` only from pods that have the label `role=frontend`.” Apply the correct manifest file.
- **Validation:** Test connectivity from a `frontend` pod to an `api` pod (should succeed) and from another pod (should fail).

## Solution

To strictly allow traffic from a specific set of pods (`role=frontend`) to another set (`role=api`) on a specific port (`8000`), the NetworkPolicy must:

1. **Target the destination pods**: Use `spec.podSelector` to select `role=api`.
2. **Define Ingress source**: Use an `ingress` rule with `from: - podSelector` matching `role=frontend`.
3. **Specify Port**: Restrict traffic to TCP port `8000`.

**Correct Manifest Structure:**

You need to look for the manifest that uses `podSelector` (not `namespaceSelector`) in the `from` section, assuming all pods are in the same namespace.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      role: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 8000
```

**Common Pitfalls in Exams:**

- **Empty PodSelector**: `podSelector: {}` selects ALL pods in the namespace.
- **NamespaceSelector**: Using `namespaceSelector` instead of `podSelector` in the `from` block would allow traffic from ALL pods in a specific namespace.

**Action:** Inspect the provided files (`cat /opt/np-1.yaml`, etc.) and apply the one that matches.

```
kubectl apply -f /opt/np-<correct>.yaml
```

---

## Question 28: Configure Pod DNS Policy

- **Scenario:** A legacy application pod needs to use a specific external DNS server instead of the cluster's CoreDNS.
- **Initial State:** A pod manifest `/opt/legacy-pod.yaml` exists.
- **Task:** Edit the pod manifest `/opt/legacy-pod.yaml`. Set the `dnsPolicy` to `None`. Add a `dnsConfig` section that specifies `8.8.4.4` in the `nameservers` list. Create the pod.
- **Validation:** Exec into the created pod and inspect the contents of `/etc/resolv.conf`. It should show `nameserver 8.8.4.4`.

### Solution

By default, Pods inherit the cluster's DNS configuration (`dnsPolicy: ClusterFirst`), meaning they use CoreDNS to resolve Service names and external domains. To force a Pod to use a specific external DNS server **only**, you must:

1. Set `dnsPolicy` to `None`.
2. Provide the `dnsConfig` manually.

**YAML Manifest:**

```
apiVersion: v1
kind: Pod
metadata:
  name: legacy-pod
spec:
  containers:
    - name: legacy-app
      image: busybox:1.28
      command: ["sleep", "3600"]
  # Important: Set policy to None so it doesn't use CoreDNS
  dnsPolicy: None
  dnsConfig:
```

```
    nameservers:
      - 8.8.4.4
    searches:
      - ns1.svc.cluster.local
      - my.dns.search.suffix
```

**Validation:**

```
kubectl apply -f /opt/legacy-pod.yaml

# Check resolv.conf inside the pod
kubectl exec legacy-pod -- cat /etc/resolv.conf
# Output should contain:
# nameserver 8.8.4.4
```

---

## Question 29: Use a LoadBalancer Service

- **Scenario:** An application needs to be exposed to the internet using a cloud provider's load balancer.
- **Initial State:** A Deployment `public-api` is running. The environment is a simulated cloud environment.
- **Task:** Create a Service named `api-lb` of type `LoadBalancer`. It should target the `public-api` pods on port `8080`.
- **Validation:** Run `kubectl get service api-lb`. After a moment, an `EXTERNAL-IP` should be assigned by the simulated cloud provider.

### Solution

A `LoadBalancer` Service exposes the Service externally using a cloud provider's load balancer. NodePort and ClusterIP services are automatically created, and the external load balancer routes to them.

**Imperative Creation:**

```
kubectl create service loadbalancer api-lb --tcp=8080:8080 --dry-run=client -o yaml >
```

*Note: The imperative command assumes a selector `app=api-lb`, so you will likely need to edit the YAML to match the actual Deployment labels.*

**YAML Manifest:**

```
apiVersion: v1
kind: Service
metadata:
  name: api-lb
spec:
  selector:
    app: public-api # Must match the deployment labels
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
  type: LoadBalancer
```

**Validation:**

```
kubectl apply -f lb.yaml
kubectl get svc api-lb
# Wait for the EXTERNAL-IP to change from <pending> to an IP address.
```

---

## Question 30: Modify CoreDNS for Custom Domains

- **Scenario:** You need CoreDNS to resolve a custom domain `internal.corp` to a specific IP address.
- **Initial State:** CoreDNS is running.
- **Task:** Edit the `coredns` ConfigMap in the `kube-system` namespace. Add a `hosts` block to the Corefile configuration that maps `service.internal.corp` to the IP `10.0.5.20`. Restart the CoreDNS pods.
- **Validation:** Exec into a test pod and run `nslookup service.internal.corp`. It should resolve to `10.0.5.20`.

### Solution

CoreDNS configuration is stored in a ConfigMap named `coredns` in the `kube-system` namespace. You can modify the `Corefile` within this ConfigMap to add custom DNS entries using the `hosts` plugin.

**Step 1: Edit the ConfigMap**

```
kubectl edit configmap coredns -n kube-system
```

Add the `hosts` block inside the main `.:53 { ... }` block:

```
    hosts {
      10.0.5.20 service.internal.corp
      fallthrough
    }
```

*Note: The format is `<IP> <HOSTNAME>`.*

**Step 2: Restart CoreDNS**

For the changes to take effect immediately, restart the CoreDNS pods.

```
kubectl rollout restart deployment coredns -n kube-system
```

**Validation:**

```
# Run a temporary pod to test resolution
kubectl run dns-test --image=busybox:1.28 --rm -it -- nslookup service.internal.corp
```

---

## Domain: Workloads & Scheduling (10 Questions)

## Question 31: Configure a Horizontal Pod Autoscaler

- **Scenario:** A CPU-intensive workload needs to scale automatically based on load.
- **Initial State:** A Deployment named `processor` is running. The pods in the deployment have CPU resource requests set.

- **Task:** Create a HorizontalPodAutoscaler named `processor-hpa`. It should target the `processor` Deployment, maintain a minimum of 2 and a maximum of 6 replicas, and scale up when the average CPU utilization across pods exceeds 60%.
- **Validation:** Run `kubectl describe hpa processor-hpa` to verify its configuration.

Here is the YAML manifest for **Question 31 – Horizontal Pod Autoscaler**:

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: processor-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: processor
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

Imperative Commands for Question 31 (Deployment + HPA)

```
kubectl autoscale deployment processor \
  --cpu-percent=60 \
  --min=2 \
  --max=6
```

---

## Question 32: Calculate and Set Resource Limits

- **Scenario:** A Deployment is causing instability by consuming too many resources on a node. You need to constrain it based on available node capacity.
- **Initial State:** A node `worker-1` has 2000m of allocatable CPU and 4Gi of allocatable memory. A Deployment `resource-hog` with 2 replicas is running on it.

- **Task:** Calculate resource limits for the `resource-hog` pods. Each pod should be limited to `400m` CPU and `512Mi` of memory. Edit the `resource-hog` Deployment and set these values in the `resources.limits` section of the container spec.
- **Validation:** Describe one of the `resource-hog` pods and verify that the CPU and memory limits are correctly set.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-hog
spec:
  replicas: 2
  selector:
    matchLabels:
      app: resource-hog
  template:
    metadata:
      labels:
        app: resource-hog
    spec:
      containers:
        - name: resource-hog
          image: your-image-here
          resources:
            limits:
              cpu: "400m"
              memory: "512Mi"
            requests:
              cpu: "400m"
              memory: "512Mi"
```

---

## Question 33: Use a PriorityClass for a Critical Pod

- **Scenario:** A critical monitoring agent must be scheduled even if the cluster is under high load.
- **Initial State:** A pod manifest `/opt/agent.yaml` exists.
- **Task:** Create a `PriorityClass` named `high-priority` with a value of `1000000`. Edit the pod manifest `/opt/agent.yaml` to use this priority class by setting the `priorityClassName` field to `high-priority`. Create the pod.

- **Validation:** Describe the created pod and verify that the **Priority Class** field is set to `high-priority`.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
description: "This priority class is for critical monitoring agents"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: critical-agent
spec:
  priorityClassName: high-priority
  containers:
    - name: agent
      image: busybox
      command: ["sh", "-c", "while true; do echo running; sleep 5; done"]
```

```
kubectl describe pod critical-agent | grep -i Priority
```

---

## Question 34: Add a Sidecar Container

- **Scenario:** An existing application pod needs a sidecar container to stream its logs to a central service.
- **Initial State:** A Deployment `main-app` is running. The application writes logs to `/var/log/app.log` inside a volume.
- **Task:** Edit the `main-app` Deployment. Add a new container to the pod spec named `log-streamer` using the `busybox` image. The new container should run the command `tail -f /var/log/app.log`. Mount the same log volume that the main application uses into this new sidecar container.
- **Validation:** Describe one of the pods from the `main-app` Deployment. It should show two containers: the main app container and the `log-streamer`.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: main-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: main-app
  template:
    metadata:
      labels:
        app: main-app
    spec:
      containers:
        - name: main-app
          image: nginx:1.17.0
          volumeMounts:
            - name: log-volume
              mountPath: /var/log/
          resources:
            requests:
              memory: "64Mi"
              cpu: "256m"
            limits:
              memory: "128Mi"
              cpu: "512m"
        - name: log-streamer
          image: busybox
          command: ["sh", "-c", "tail -f /var/log/app.log"]
          volumeMounts:
            - name: log-volume
              mountPath: /var/log/
          resources:
            requests:
              memory: "64Mi"
              cpu: "256m"
            limits:
              memory: "128Mi"
              cpu: "512m"
      volumes:
        - name: log-volume
          emptyDir: {}
```

---

## Question 35: Perform a Rolling Update and Rollback

- **Scenario:** You need to update an application to a new version and then roll it back due to a reported bug.
- **Initial State:** A Deployment `frontend` is running version `1.0` of an application image.
- **Task:** Update the `frontend` Deployment to use image version `1.1`. After the update completes, perform a rollback to the previous version.
- **Validation:** Run `kubectl rollout history deployment frontend` to see the revision history. After the rollback, describe the deployment and verify that it is using image version `1.0` again.

To Update:

```
kubectl set image deployment/frontend frontend-container=your-image:1.1
kubectl rollout status deployment/frontend
```

To Rollback:

```
kubectl rollout undo deployment/frontend
kubectl rollout status deployment/frontend
```

To Validate:

```
kubectl rollout history deployment frontend
kubectl describe deployment frontend | grep Image
```

---

## Question 36: Use a ConfigMap to Configure an Application

- **Scenario:** An NGINX pod needs to be configured with a custom `nginx.conf` file.
- **Initial State:** A file `/opt/nginx.conf` exists with custom settings.
- **Task:** Create a ConfigMap named `nginx-config` from the file `/opt/nginx.conf`. Then, create a pod that runs the `nginx` image. Mount the `nginx-config` ConfigMap as a volume into the pod at the path `/etc/nginx/nginx.conf`, overwriting the default configuration file.

- **Validation:** Exec into the pod and run `cat /etc/nginx/nginx.conf`. The content should match the custom file from `/opt/nginx.conf`.

Solution for Question 36: Use a ConfigMap to Configure an Application

### Step 1: Create the ConfigMap from the file

```
kubectl create configmap nginx-config --from-file=nginx.conf=/opt/nginx.conf
```

### Step 2: Create the Pod Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-custom
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: nginx-config-volume
          mountPath: /etc/nginx/nginx.conf
          subPath: nginx.conf
  volumes:
    - name: nginx-config-volume
      configMap:
        name: nginx-config
        items:
          - key: nginx.conf
            path: nginx.conf
```

### Step 3: Apply the Pod Manifest

```
kubectl apply -f nginx-pod.yaml
```

### Validation

```
kubectl exec -it nginx-custom -- cat /etc/nginx/nginx.conf
```

You should see the contents of `/opt/nginx.conf`.

---

## Question 37: Use a Secret for Environment Variables

- **Scenario:** An application requires a database password, which must be supplied securely as an environment variable.
- **Initial State:** A pod manifest `/opt/app-pod.yaml` exists.
- **Task:** Create a generic Secret named `db-secret` with a key `DB_PASSWORD` and a value of `s3cr3tP@ssw0rd`. Edit the pod manifest `/opt/app-pod.yaml` to expose the `DB_PASSWORD` key from the `db-secret` as an environment variable named `DATABASE_PASSWORD` in the container. Create the pod.
- **Validation:** Exec into the pod and run `env | grep DATABASE_PASSWORD`. It should show `DATABASE_PASSWORD=s3cr3tP@ssw0rd`.

Solution for Question 37: Use a Secret for Environment Variables

### Step 1: Create the Secret

```
kubectl create secret generic db-secret \
  --from-literal=DB_PASSWORD=s3cr3tP@ssw0rd
```

### Step 2: Edit `/opt/app-pod.yaml` to include the Secret as an environment variable

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app-container
      image: your-application-image
      env:
        - name: DATABASE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: DB_PASSWORD
```

#### Step 3: Apply the manifest

```
kubectl apply -f /opt/app-pod.yaml
```

#### Validation

```
kubectl exec -it app-pod -- env | grep DATABASE_PASSWORD
```

Expected output:

```
DATABASE_PASSWORD=s3cr3tP@ssw0rd
```

---

## Question 38: Configure Node Affinity

- **Scenario:** A specific workload must only run on nodes that have high-performance SSD storage.
- **Initial State:** Some worker nodes have the label `disktype=ssd`. A Deployment manifest `/opt/workload.yaml` exists.
- **Task:** Edit the Deployment manifest `/opt/workload.yaml`. Add a `nodeAffinity` rule under `spec.template.spec.affinity`. The rule should be a `requiredDuringSchedulingIgnoredDuringExecution` type that requires nodes to have the label `disktype` with the value `ssd`. Apply the manifest.
- **Validation:** Check which nodes the deployment's pods are running on using `kubectl get pods -o wide`. All pods should be on nodes with the `disktype=ssd` label.

```
k label node cka-multinode-m03 disktype=ssd
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-ssd-affinity
spec:
  replicas: 3
  selector:
```

```
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx
          name: blue
          resources:
            requests:
              memory: "64Mi"
              cpu: "256m"
            limits:
              memory: "128Mi"
              cpu: "512m"
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: disktype
                    operator: In
                    values:
                      - ssd
```

---

## Question 39: Configure Taints and Tolerations

- **Scenario:** A specific node is reserved for GPU workloads and should not accept normal pods.
- **Initial State:** A node `gpu-node-1` exists. A pod manifest `/opt/gpu-pod.yaml` exists.
- **Task:** Add a taint to the `gpu-node-1` node with the key `gpu`, value `true`, and effect `NoSchedule`. Then, edit the pod manifest `/opt/gpu-pod.yaml` to add a toleration for this taint (`key: "gpu"`, `operator: "Exists"`, `effect: "NoSchedule"`). Create the pod.
- **Validation:** The `gpu-pod` should be successfully scheduled on `gpu-node-1`. A normal pod without the toleration should not be scheduled on that node.

```
# Add taint to the node
kubectl taint nodes gpu-node-1 gpu=true:NoSchedule
```

Edit the pod manifest /opt/gpu-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  tolerations:
    - key: "gpu"
      operator: "Exists"
      effect: "NoSchedule"
  containers:
    - name: gpu-container
      image: nginx
```

```
# Create the pod
kubectl apply -f /opt/gpu-pod.yaml
```

**Validation**

```
kubectl get pods -o wide
kubectl describe pod gpu-pod
```

**The Issue: Taints vs. Affinity You are assuming that a Toleration attracts a Pod to a specific Node. It does not.

Taints are Repulsive: They say "Do not come here unless you have a pass." Tolerations are Permission Slips: They say "I have a pass to enter if I need to," but they do not say "I must go there." Affinity is Attractive: This is what says "I want to go to this specific node."

---

## Question 40: Create a StatefulSet

- **Scenario:** Deploy a stateful application that requires stable network identifiers and persistent storage.
- **Initial State:** A headless service `app-headless` and a StorageClass `fast-storage` exist.
- **Task:** Create a StatefulSet named `data-app` with 2 replicas. It should use the `app-headless` service for its `serviceName`. The pod template should use the `nginx` image.

Define a `volumeClaimTemplate` that creates a 1Gi PVC for each replica using the `fast-storage` StorageClass.

- **Validation:** Verify that two pods, `data-app-0` and `data-app-1`, are created and running. Verify that two corresponding PVCs have also been created and bound.

---

## Domain: Storage (10 Questions)

## Question 41: Create a PVC and Mount it in a Pod

- **Scenario:** An application needs a persistent volume to store its data.
- **Initial State:** A default StorageClass is configured in the cluster.
- **Task:** Create a PersistentVolumeClaim named `my-pvc` that requests `1Gi` of storage with the `ReadWriteOnce` access mode. Then, create a pod named `storage-pod` that mounts this PVC at the path `/data`.
- **Validation:** Exec into the `storage-pod` and create a file in the `/data` directory. Delete the pod. Recreate the pod. Exec into the new pod and verify that the file still exists in `/data`.

### Solution

This question tests your ability to dynamically provision storage using a PersistentVolumeClaim (PVC) and mount it into a Pod. Since a default StorageClass exists, you only need to create the PVC, and the cluster will automatically create the backing PV.

**1. Create the PersistentVolumeClaim (PVC):**

Create a file named `my-pvc.yaml`:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
```

```
    requests:
      storage: 1Gi
```

### 2. Create the Pod:

Create a file named `storage-pod.yaml`. Note the `volumes` and `volumeMounts` sections.

```
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
spec:
  containers:
    - name: busybox
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - mountPath: "/data"
          name: my-data
  volumes:
    - name: my-data
      persistentVolumeClaim:
        claimName: my-pvc
```

### 3. Apply the Manifests:

```
kubectl apply -f my-pvc.yaml
kubectl apply -f storage-pod.yaml
```

### 4. Validation:

Verify the PVC is bound:

```
kubectl get pvc my-pvc
# STATUS should be 'Bound'
```

Test Data Persistence:

```
# 1. Write data
kubectl exec storage-pod -- sh -c "echo 'Hello Storage' > /data/file.txt"
```

```
# 2. Delete the pod (data stays in PVC/PV)
kubectl delete pod storage-pod

# 3. Recreate the pod
kubectl apply -f storage-pod.yaml

# 4. Verify data exists
kubectl exec storage-pod -- cat /data/file.txt
# Output should be 'Hello Storage'
```

**Note on Physical Storage (Minikube):** Since you are using the default `standard` StorageClass in Minikube, it uses the `hostPath` provisioner. The actual files are stored on the Minikube node (not your local laptop's filesystem) at a path like `/tmp/hostpath-provisioner/default/my-pvc`. You can verify this by SSHing into the node (`minikube ssh`) and checking that directory.

---

## Question 42: Patch a StorageClass to be the Default

- **Scenario:** The cluster has two StorageClasses, `slow` and `fast`, but neither is the default. You need to make `fast` the default for all new PVCs.
- **Initial State:** Two StorageClasses, `slow` and `fast`, exist.
- **Task:** Use the `kubectl patch` command to add the annotation `storageclass.kubernetes.io/is-default-class="true"` to the `fast` StorageClass. Ensure the `slow` StorageClass does not have this annotation.
- **Validation:** Create a new PVC without specifying a `storageClassName`. Describe the PVC and verify that it was provisioned using the `fast` StorageClass.

### Solution

To set a StorageClass as the default, you need to add the specific annotation `storageclass.kubernetes.io/is-default-class: "true"`. This can be done using the `kubectl patch` command.

**1. Check Current StorageClasses:**

```
kubectl get sc
# You should see 'slow' and 'fast' (and likely 'standard').
```

**2. Patch 'fast' to be Default:**

```
kubectl patch storageclass standard -p '{"metadata": {"annotations":{"storageclass.kub
```

*Note: If another class (like `standard`) is already marked as default, you might want to remove its default status to avoid confusion, though Kubernetes usually picks the one most recently marked or warns if multiple exist.*

**3. Ensure 'slow' is NOT Default:**

If `slow` was previously default, you would remove the annotation or set it to false:

```
kubectl patch storageclass slow -p '{"metadata": {"annotations":{"storageclass.kuberne
```

**4. Validation:**

Verify the output of `get sc`:

```
kubectl get sc
# The 'fast' class should now display '(default)' next to its name.
```

Create a PVC without specifying a ConfigMap:

```
# test-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-default-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
```

```
kubectl apply -f test-pvc.yaml
kubectl get pvc test-default-pvc
```

```
kubectl describe pvc test-default-pvc | grep "StorageClass"
# Should show: StorageClass: fast
```

**Common Issue: Pending PVC** If the PVC remains in `Pending` state with "no persistent volumes available for this claim and no storage class is set":

1. **Cause**: The PVC yaml likely has `storageClassName: ""` which explicitly disables dynamic provisioning.
2. **Fix**: Remove the `storageClassName` field (or set it to `null`) to allow the default StorageClass to take over.
3. **Note**: PVC `spec` is immutable. You cannot edit the field on an existing PVC. **You must delete and recreate the PVC.**

---

## Question 43: Create a PVC to Bind to a Specific PV

- **Scenario:** A PersistentVolume named `pv-data-001` was created manually, and you need to create a claim that specifically binds to it.
- **Initial State:** A PV named `pv-data-001` exists with a capacity of `2Gi`, access mode `ReadWriteOnce`, and `storageClassName` of `manual`.
- **Task:** Create a PersistentVolumeClaim named `claim-for-pv-data` that exactly matches the specifications of `pv-data-001` (requests 2Gi storage, ReadWriteOnce access mode, and `storageClassName: manual`) to ensure it binds to that specific PV.
- **Validation:** Run `kubectl get pvc claim-for-pv-data`. Its status should be `Bound`, and describing it should show it is bound to the `pv-data-001` volume.

### Solution

This question tests your understanding of static provisioning and how PVCs bind to specific PVs based on matching criteria (Capacity, Access Mode, and Storage Class).

**Step 1: Setup - Create the Initial PV (Pre-requisite)** First, create the Persistent Volume as described in the "Initial State".

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolume
metadata:
```

```
  name: pv-data-001
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  hostPath:
    path: "/tmp/data-001"
EOF
```

**Step 2: Create the PVC** Create a PVC that requests specific resources matching the PV. Crucially, the `storageClassName` must match.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: claim-for-pv-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
  storageClassName: manual
EOF
```

**Step 3: Validation**

Check if the PVC is bound to the correct PV.

```
# Check status - should be "Bound"
kubectl get pvc claim-for-pv-data

# Verify it is bound to 'pv-data-001'
kubectl describe pvc claim-for-pv-data | grep "Volume:"
# Output should show: Volume: pv-data-001
```

---

## Question 44: Configure Volume Reclaim Policy

- **Scenario:** You are creating a PersistentVolume for temporary data and want the underlying storage to be deleted when the claim is released.
- **Initial State:** A manifest for a PV is at `/opt/pv.yaml`.
- **Task:** Edit the PV manifest at `/opt/pv.yaml`. Set the `persistentVolumeReclaimPolicy` field to `Delete`. Create the PV.
- **Validation:** Create a PVC that binds to this PV. Then, delete the PVC. The PV's status should change to `Terminating` and it should eventually be deleted.

## Solution

The `persistentVolumeReclaimPolicy` controls what happens to a PersistentVolume when its bound PVC is deleted.

- **Retain**: (Default for manual PVs) The PV is released but keeps the data. It requires manual cleanup.
- **Delete**: (Default for dynamic provisioning) The PV is deleted along with the underlying storage.
- **Recycle**: (Deprecated) Performs a basic scrub (rm -rf).

**Step 1: Define the PV with Reclaim Policy 'Delete'**

Create or edit `/opt/pv.yaml`.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: temp-pv
spec:
  capacity:
    storage: 100Mi
  accessModes:
    - ReadWriteOnce
  # This is the key setting
  persistentVolumeReclaimPolicy: Delete
  hostPath:
    path: /tmp/temp-data
```

Apply it:

```
kubectl apply -f /opt/pv.yaml
```

**Step 2: Create a PVC to bind to it**

To test it, we need a PVC that will bind to this specific PV. We can use a selector or just match the attributes.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: temp-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
  # Ensure this is empty to match the PV if it doesn't have a class,
  # or match the PV's class if it has one.
  storageClassName: ""
```

**Step 3: Validation (The Delete Test)**

```
# 1. Create resources
kubectl apply -f /opt/pv.yaml
kubectl apply -f temp-pvc.yaml

# 2. Verify binding
kubectl get pv temp-pv
# Status should be Bound

# 3. Delete the PVC
kubectl delete pvc temp-pvc

# 4. Check the PV again
kubectl get pv temp-pv
# It should be gone (or briefly Terminating)
```

---

## Question 45: Expand a Persistent Volume Claim

- **Scenario:** An application's database is running out of space, and the underlying storage volume needs to be expanded.
- **Initial State:** A StorageClass `expandable-sc` with `allowVolumeExpansion: true` exists. A PVC `db-pvc` created with this class is currently `10Gi`.
- **Task:** Edit the `db-pvc` PersistentVolumeClaim and change the `spec.resources.requests.storage` value from `10Gi` to `20Gi`.
- **Validation:** Run `kubectl describe pvc db-pvc`. Check the events to see a successful resize operation. The capacity of the PVC should now show `20Gi`.

## Solution

This question tests your ability to expand Persistent Volumes. This feature must be enabled in the StorageClass via `allowVolumeExpansion: true`. The minikube `standard` class usually has this enabled (or we can patch it).

### Step 1: Setup - Create StorageClass and PVC (Pre-requisite)

```
# 1. Create a StorageClass that allows expansion
cat <<EOF | kubectl apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: expandable-sc
provisioner: k8s.io/minikube-hostpath
allowVolumeExpansion: true
EOF

# 2. Create the initial PVC
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: expandable-sc
  resources:
    requests:
```

```
      storage: 10Gi
EOF
```

**Step 2: Expand the PVC**

You can do this via `kubectl edit` or patching.

```
kubectl patch pvc db-pvc -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'
```

Alternatively, open the editor:

```
kubectl edit pvc db-pvc
# Change storage: 10Gi to 20Gi
```

**Step 3: Validation**

```
# Check the status
kubectl get pvc db-pvc
# Look for CAPACITY 20Gi (It usually happens instantly on hostPath)

# Check events for confirmation
kubectl describe pvc db-pvc
# Look for: FileSystemResizeForSuccessful
```

> ***Note**: For some storage types, the resize only happens after a Pod using the PVC is restarted or the filesystem is mounted. For `hostPath` (minikube), it is usually immediate.*

---

## Question 46: Use a HostPath Volume

- **Scenario:** A pod needs to access logs stored directly on the node's filesystem for debugging.
- **Initial State:** A log file exists at `/var/log/node-app.log` on a worker node.
- **Task:** Create a pod that uses a `hostPath` volume to mount the `/var/log` directory from the host node into the pod at the path `/node-logs`. The pod should be scheduled to the specific worker node where the log file exists.
- **Validation:** Exec into the pod and run `cat /node-logs/node-app.log`. It should display the contents of the log file from the host node.

## Solution

This question covers mounting a directory from the underlying node into a Pod using `hostPath`. This is often used for system agents or debugging but has security implications (the Pod can read/write host files).

### Step 1: Identify the Node

Find the node name you want to schedule on.

```
kubectl get nodes
# Assume node is 'worker-node-1' / 'minikube'
```

### Step 2: Create the Pod Manifest

Use `nodeName` to force scheduling to that specific node.

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  # Force scheduling to the node where the log file is
  nodeName: minikube # Change this to your actual node name
  containers:
    - name: logs-viewer
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - mountPath: /node-logs
          name: log-volume
  volumes:
    - name: log-volume
      hostPath:
        path: /var/log
        type: Directory
```

### Step 3: Validation

First, ensure the dummy file exists on the node (Minikube example):

```
minikube ssh "sudo touch /var/log/node-app.log && echo 'Host Log Content' | sudo tee /
```

Then create and check the pod:

```
kubectl apply -f hostpath-pod.yaml

# Read the file from inside the pod
kubectl exec hostpath-pod -- cat /node-logs/node-app.log
# Output: Host Log Content
```

---

## Question 47: Use an emptyDir Volume for Temporary Data

- **Scenario:** Two containers in a pod need to share temporary files.
- **Initial State:** A pod manifest `/opt/multi-container-pod.yaml` defines two containers.
- **Task:** Edit the pod manifest. Define an `emptyDir` volume named `shared-data`. Mount this volume into both containers at the path `/shared`. Create the pod.
- **Validation:** Exec into the first container and create a file in `/shared`. Then, exec into the second container and verify that the file is visible and accessible at `/shared`.

### Solution

This scenario demonstrates the sidecar pattern where containers in the same Pod share a filesystem. `emptyDir` creates a temporary directory that exists as long as the Pod is running.

#### Step 1: Create the Pod Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-pod
spec:
  containers:
    - name: c1
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - name: shared-data
```

```
          mountPath: /shared
    - name: c2
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /shared
  volumes:
    - name: shared-data
      emptyDir: {}
```

**Step 2: Apply and Validate**

```
# Apply
kubectl apply -f multi-container-pod.yaml

# Create a file in container 1
kubectl exec multi-container-pod -c c1 -- sh -c "echo 'Shared Data' > /shared/data.txt"

# Read the file from container 2
kubectl exec multi-container-pod -c c2 -- cat /shared/data.txt
# Output should be: Shared Data
```

---

## Question 48: Create a Read-Only Volume Mount

- **Scenario:** A pod needs to read configuration data from a ConfigMap, but it must be prevented from modifying the data.
- **Initial State:** A ConfigMap `app-config` exists.
- **Task:** Create a pod. Mount the `app-config` ConfigMap as a volume. In the `volumeMounts` section for the container, set the `readOnly` field to `true`.
- **Validation:** Exec into the pod and attempt to write a file to the mounted config directory (e.g., `touch /path/to/config/new-file`). The command should fail with a "Read-only file system" error.

### Solution

Mounting volumes as `readOnly` is a best practice for configuration data (ConfigMaps/Secrets) to ensure immutability from the application's perspective.

### Step 1: Create the ConfigMap (Pre-requisite)

```
kubectl create configmap app-config --from-literal=config.json='{"debug":true}'
```

### Step 2: Create the Pod with ReadOnly Mount

```
apiVersion: v1
kind: Pod
metadata:
  name: readonly-pod
spec:
  containers:
    - name: app
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
          readOnly: true
  volumes:
    - name: config-volume
      configMap:
        name: app-config
```

### Step 3: Validation

```
kubectl apply -f readonly-pod.yaml

# Try to write to the mount path
kubectl exec readonly-pod -- touch /etc/config/hack.txt
# Output should be: touch: /etc/config/hack.txt: Read-only file system
```

---

## Question 49: Manually Create a PersistentVolume

- **Scenario:** You have an existing NFS share and need to make it available as a volume in the cluster.
- **Initial State:** An NFS server is running at `192.168.1.100` with an exported path `/data/shared`.

- **Task:** Create a PersistentVolume manifest. The PV should have a capacity of `5Gi`, access mode `ReadWriteMany`, and a `reclaimPolicy` of `Retain`. The volume source should be `nfs`, with the server set to `192.168.1.100` and the path to `/data/shared`. Apply the manifest.
- **Validation:** Run `kubectl get pv`. The new PV should be listed with the status `Available`.

### Solution

This tests your ability to create static PersistentVolumes, specifically for Network File Systems (NFS), which support `ReadWriteMany` (RWX) access modes.

#### Step 1: Create the PV Manifest

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    server: 192.168.1.100
    path: "/data/shared"
```

#### Step 2: Apply and Validate

```
kubectl apply -f nfs-pv.yaml

# Check the status
kubectl get pv nfs-pv
# Status should be 'Available' (since no PVC has bound to it yet)
```

---

## Question 50: Clone a Persistent Volume Claim

- **Scenario:** You need to create a pre-populated volume for a new testing environment by cloning an existing PVC.
- **Initial State:** A PVC `source-pvc` exists and is bound. The underlying CSI driver supports volume cloning.
- **Task:** Create a new PVC manifest for a PVC named `cloned-pvc`. In the `spec`, add a `dataSource` block that specifies `name: source-pvc` and `kind: PersistentVolumeClaim`. Apply the manifest.
- **Validation:** Run `kubectl get pvc cloned-pvc`. It should become `Bound`. Create a pod that mounts `cloned-pvc`, and verify that it contains the same data as the `source-pvc`.

## Solution

Volume cloning allows you to provision a new PersistentVolumeClaim with data populated from an existing PVC. This is handled by the CSI driver (assuming it supports cloning). The key configuration is the `dataSource` field.

> ***Note**: The source PVC must be in the same namespace and have the same StorageClass (CSI driver).*

### Step 1: Create the Clone PVC Manifest

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cloned-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi # Must be >= source PVC size
  storageClassName: fast # Must match source or be a compatible CSI class
  dataSource:
    kind: PersistentVolumeClaim
    name: source-pvc
```

### Step 2: Apply and Validate

```
# 1. Apply the manifest
kubectl apply -f cloned-pvc.yaml

# 2. Check the status
kubectl get pvc cloned-pvc
# Status should be 'Bound'

# 3. Validation Pod
# Create a pod to mount the new volume and inspect contents
kubectl run validator --image=busybox --restart=Never --command -- sleep 3600
# (Mounting requires a full manifest or overrides, typically you would just inspect the
```